

Extended Abstract

Motivation Starting from Qwen 2.5 0.5B Base model, we fine-tuned it with SFT and RLOO. While the RLOO offered some performance gain over the SFT baseline, it suffered from two specific issues. The model tended to generate excessively long reasoning strings within the `<think>` clause, and a large portion of its incorrect answers stemmed from using redundant numbers in the equation. This verbosity was particularly problematic, as it often caused the final answer section to be truncated by the output limit, hindering the model from receiving a direct signal from the reward function. To address these issues while preserving current performance, we proposed a **Curriculum Based Reward Adjustment** strategy.

Method Our approach aims to guide the model to slowly truncate its chain of thought and output only the final answer with the `<answer>` tag. In addition, we implemented a direct penalty to penalize the model whenever it produced redundant numbers. Our methodology involved a sequential fine-tuning process starting from the Qwen2-0.5B-Base model. We first established a baseline with SFT using the Warmstart dataset and then employed RLOO to enhance performance. To address specific generative flaws observed after this stage, we shaped the reward mechanism with two main goals: to guide the model toward truncating its chain of thought and to directly penalize the use of redundant numbers.

Implementation We implemented our reward function as described in Algorithm 1, which modifies the base RLOO reward with terms for conciseness and correctness. We trained with the new reward algorithm starting from our best-performing RLOO model using unseen data from the Countdown dataset. However, a key limitation was discovered during training: the model’s running correctness ratio never surpassed the 0.4 threshold required to advance to the second stage of the curriculum. Consequently, the penalty for redundant numbers (stage 2 and 3) remained unexplored. All experiments were conducted on an AWS EC2 instance with an NVIDIA L40S GPU.

Results Our method led to a significant and paradoxical outcome. The curriculum-trained model achieved a final leaderboard score of **0.3079**, an **18%** accuracy improvement over the RLOO baseline score of 0.2620. However, contrary to our hypothesis, the model’s average output length increased by approximately 150 tokens (from 979.0 to 1130.9). This increase was statistically significant with $p \ll 0.001$ using Wilcoxon Signed-Rank test.

Discussion We hypothesized that rewarding brevity would shorten model outputs and yield neutral or positive effects on accuracy. Contrary to this expectation, the curriculum *increased* accuracy while *lengthening* the average chain-of-thought. These results show that our reward design changed behavior, but not in the direction we intended. Two factors likely account for this divergence. First, the unit reward for a correct answer overwhelms length bonus, so gradient updates inevitably prioritize correctness at the expense of conciseness. As a result, the model *hacks* the objective by pursuing the dominant payoff and ignoring the weaker signal. Second, explicit reasoning tokens might act as scratch-pad memory for a 0.5B parameter model trying to tackle the combinatorial Countdown search. Removing that scaffold would raise the risk of failure, so the optimizer settles on longer explanations that more reliably reach the target number.

Conclusion While our method successfully improved task accuracy, it paradoxically increased solution length due to a *reward hacking* dynamic where the model prioritized the large reward for correctness over the smaller bonus for brevity. This finding sets a clear path for future work. Our next steps will focus on re-calibrating the reward function by systematically adjusting hyperparameters and exploring "harder" constraints, such as significant length penalties or a multi-objective optimization framework. Once we have a model that reliably meets the threshold accuracy, we will apply a final curriculum stage specifically designed to minimize its tendency to repeat numbers, thereby addressing all identified flaws in a sequential manner.

Curriculum Based Reward Adjustment for CoT Internalization

Kyung Geun Kim
Department of Bioengineering
Stanford University
kyunggk@stanford.edu

Jinho So
Department of Mechanical Engineering
Stanford University
jinhoso@stanford.edu

Abstract

Refining fine-tuned language models often leads to trade-offs between task accuracy and desirable generative behaviors. In this work, we tackle two such flaws—excessive verbosity and input redundancy—that emerged after fine-tuning a Qwen2-0.5B model for the Countdown numerical reasoning task. We propose Curriculum Based Reward Adjustment, a reward-shaping method that provides a bonus for concise solutions and penalizes errors in stages. While our approach successfully improved overall task accuracy by 18% compared to a strong RLOO baseline, it unexpectedly led to longer, more verbose outputs. This paradoxical result suggests a *reward hacking* dynamic, where the model prioritizes the dominant reward signal for correctness over the weaker incentive for brevity. Our findings provide a key insight into the complexities of reward engineering, demonstrating that secondary objectives require carefully calibrated incentives to effectively influence model behavior.

1 Introduction

Large language models (LLMs) have evolved far beyond simple prompt-response systems and now drive tasks ranging from visual perception Zhu et al. (2023) and novel protein design Ferruz and Höcker (2022) to robotic planning Singh et al. (2023), and quantitative finance Lee et al. (2025), serving as versatile reasoning engines rather than mere text generators. Although larger models typically achieve stronger baseline performance, a smaller backbone that is rigorously fine-tuned for a specific domain can surpass far bigger, general-purpose LLMs—delivering equal or better task accuracy at a fraction of the computational cost Kim et al. (2024); Zhang et al. (2025).

In this context, iteratively refining a model by applying further fine-tuning on top of an already fine-tuned model is a promising method. It is advantageous as it saves the computational cost of re-training from scratch and allows for targeted improvements while preserving the model’s existing capabilities. However, this approach is vulnerable to catastrophic forgetting French (1999), a phenomenon where the model abruptly loses previously learned knowledge upon learning a new task. This presents a significant obstacle to continuous model development.

To address this, we propose Curriculum Based Reward Adjustment, a method that gently refines the reward function to steer a fine-tuned model toward a desired behavior. While curriculum learning—presenting easier examples before more difficult ones—is a popular technique for improving training stability Bengio et al. (2009), it is typically applied to the training data. We, however, adapt this concept to the reward model itself. This approach has two key advantages. First, it enables a more controlled, gradual adaptation of the model’s behavior, mitigating the risk of catastrophic forgetting. Second, it offers a direct mechanism to penalize undesirable output characteristics, such as excessive length, without altering the dataset.

To validate our method, we chose the Countdown problem, a numerical reasoning game where the goal is to use a given set of numbers and basic arithmetic operations to reach a target number. As instructed in the guideline, we first performed Supervised Fine-Tuning (SFT) on a small language model, Qwen 2.5 0.5B Base, followed by fine-tuning with REINFORCE Leave One-Out (RLOO). While RLOO improved performance by about 9% over SFT, it introduced new errors: the model began to utilize the same numbers within a solution which is a violation of the Countdown rules and generate excessively long chains of thought, often failing to produce the final `<answer>` tag within the output limit. The naive solution that simply increasing the maximum output length was infeasible as it would make the training process prohibitively slow. To refine this, we implemented our reward shaping strategy by adding a length constraint to the reward function.

Our application of a reward curriculum led to a significant improvement in the model’s overall task accuracy. While this method did not produce more concise answers as initially hypothesized, it provided a crucial insight into the complex dynamics of reward shaping. The primary contributions of this work are:

- We demonstrate an effective workflow for iterative fine-tuning on a specialized mathematical reasoning task, progressing from SFT to RLOO and finally to reward shaping.
- We introduce Curriculum-Based Reward Adjustment, a novel reward-shaping technique designed to address specific generative flaws in a fine-tuned model.
- We present a key empirical finding on the complexities of reward shaping. We show that while our method successfully improved task accuracy, it paradoxically led to an increase in output verbosity, highlighting a *reward hacking* dynamic where primary incentives can overshadow secondary ones.

2 Related Work

2.1 Curriculum learning for LLM

Curriculum learning is a training strategy that introduces examples to a model in a meaningful order, gradually presenting new concepts of increasing complexity Bengio et al. (2009). Its goal is to help models learn faster, train more stably, and ultimately achieve better performance on test data.

Recently, various researches attempts to apply it to the LLM fine-tuning. Xu et al. (2020) introduced curriculum learning to NLU fine-tuning. They first split the training set into N meta-datasets, trained a separate “teacher” model on each split, and then measured each example’s difficulty by aggregating how well the other teachers predicted it. Finally, they fine-tuned the main model in stages, starting from the easiest examples and gradually introducing harder ones.

Kim and Lee (2024) propose three difficulty metrics, prompt length, attention scores, and initial pass loss, and fine-tune LLMs by progressively ordering data from easiest to hardest according to each metric. Their experiments show that curriculum-based training outperforms random data shuffling, with attention-based ordering consistently delivering the largest gains.

Campos (2021) test eight data-ordering heuristics while pre-training ELMo on WikiText-2/103; none beat standard shuffling, and random ordering matches or exceeds the hand-crafted curricula. In contrast to the gains reported by Kim and Lee (2024), this negative result leaves the effect of data order on LLMs unsettled. It also motivates our alternative reward-based curriculum, which reshapes the objective during RL fine-tuning rather than re-ordering the data.

2.2 Reward shaping

Reward shaping is a popular technique for accelerating model training and mitigating sparse-reward environments, first proposed by Ng et al. (1999). By adding artificial rewards on top of the original reward signal, the method steers the agent toward more desirable behaviors and speeds up learning. For example, consider an agent navigating a grid world where it receives a reward only upon reaching the goal. We can add a shaping reward proportional to the distance between the agent and the goal, giving higher rewards as the agent moves closer.

Several studies have shown that reward shaping can further boost a pre-trained model’s performance. Agarwal et al. (2025) tackle text-to-JSON structuring with a 1.5B model that was first supervised-

fine-tuned but still produced schema errors. They add a lightweight reward-shaping stage, using Group Relative Policy Optimization with two signals: (i) a soft key–value alignment score and (ii) a binary check for the required `<think>/<answer>` blocks. This extra RL pass virtually eliminates format mistakes and noise, allowing the compact “ThinkJSON” model to surpass both SFT-only distilled DeepSeek variants and much larger baselines—showing how targeted reward shaping can refine an already fine-tuned model without heavy retraining or human labels. The structured-text task targeted in this study closely mirrors the Countdown task we are tackling, giving us strong inspiration and motivation for our own work.

3 Method

Our approach follows a sequential fine-tuning process, beginning with the Qwen2-0.5B-Base model. We first established a baseline with SFT. To enhance performance beyond this baseline, we then employed RLOO. Finally, to address specific generative flaws, we shaped the reward mechanism by refining the original Countdown scoring function. Each phase of this process is detailed in the following sections.

3.1 Supervised Fine-Tuning (SFT)

SFT is standard supervised learning step where we take a pretrained language model and further fine-tune it on high quality data like human annotated data using the same next-token prediction objective as pretraining:

$$\max_{\theta} \mathbb{E}_{(x,y) \in \mathcal{D}} \left[\sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}) \right]$$

We selected the Countdown task to evaluate whether the reinforcement learning improves a model performance. In this project, SFT was used to produce a fine-tuned reference model. We trained it on a well-defined dataset that includes both chain of thought reasoning and the corresponding correct answer.

3.2 REINFORCE Leave-One-Out (RLOO)

RLOO is an on-policy policy-gradient method that reduces variance and stabilizes training. Since the Countdown task requires a specific answer format, we expect RLOO to reinforce the model’s ability to generate properly formatted outputs. For each prompt x we sample k responses $y^{(1:k)} \sim \pi_{\theta}(\cdot \mid x)$, score them with a reward model $R(\cdot)$, and update with the objective:

$$\frac{1}{k} \sum_{i=1}^k \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j=1}^k R(y_{(j)}, x) \right] \nabla \log \pi_{\theta}(y_{(i)} \mid x) \quad \text{for } y_{(1)}, \dots, y_{(k)} \stackrel{i.i.d.}{\sim} \pi_{\theta}(\cdot \mid x)$$

In this project, we used the TinyZero scoring function Pan et al. (2025) for the reward model. Subtracting the average reward of the *other* samples makes each gradient term reflect only how much better (or worse) that trajectory is than its peers, sharply reducing variance compared to vanilla REINFORCE. Empirically, this stabilizes training on language tasks, allowing larger learning-rates and faster convergence without the heavy KL penalties used in policy optimization.

3.3 Curriculum Based Reward Adjustment

3.3.1 Motivation

While the RLOO implementation offers some performance gain from SFT baseline, it suffered from issues listed below.

1. During RLOO training, the model produced too much reasoning strings with `<think>` `</think>` clause.

2. A significant portion of the incorrect answers stemmed from redundant numbers.

Due to the first issue, the model is hindered from getting a direct signal from the reward function because the final answer section is often cutoff resulting in bad reward signal. In addition, the model spends too much of its capacity trying to generate reasoning strings.

To address these issues, we propose a curriculum learning strategy with reward adjustment that encourages answer formatting: we encourage the model to slowly truncate its chain of thought and output only the final answer with `<answer>` tag. In addition, we directly penalize the model if it produces redundant numbers in the equation.

This approach offers two advantages. First, all gradient signal is concentrated on generating a correct, well-formatted answer rather than on writing a lengthy explanation, so the model should better follow the required rubric. Second, omitting the explanation shortens the generated sequence, letting the model spend its fixed compute budget on the most relevant tokens—the prompt and the answer itself.

3.3.2 Reward Implementation Detail

With experiments, we discovered producing shorter output string is easier for the model to learn than ensuring single use of each number. This is because having redundant numbers in the solution is directly related to correctness of the answers. However, producing shorter CoT string can be independent to the correctness of the final answer. To account for this difference in difficulty we designed following curriculum reward algorithm 1.

Algorithm 1 CURRICULUMREWARD

Inputs:

- s solution string produced by the model
- t target value that the equation must equal
- N set of numbers supplied in the prompt that should be used 1 time
- r running correctness ratio
- α bonus reward for correct short answers ($\alpha > 0$)
- β bonus reward for short *incorrect* answers in stages 2–3
- γ penalty for duplicated numbers in stage 3

```

1: function CURRICULUMREWARD( $s, t, N, r$ )
2:    $R \leftarrow \text{BASEREWARD}(s, t, N)$  ▷ 0, 0.1, or 1
3:    $\ell \leftarrow |s|$  ▷ string length
4:    $c \leftarrow \text{ISCORRECT}(s, t)$  ▷ 1 if correct
5:    $d \leftarrow \text{HASDUPLICATES}(s)$  ▷ 1 if any number used twice
6:    $k \leftarrow \begin{cases} 1, & r < 0.4 \\ 2, & 0.4 \leq r < 0.6 \\ 3, & r \geq 0.6 \end{cases}$  ▷ curriculum stage
7:    $R \leftarrow R + c \frac{\alpha}{\ell} + \mathbf{1}\{k > 1\} (1 - c) \frac{\beta}{\ell} - \mathbf{1}\{k = 3\} d \gamma$ 
8:   return  $R$ 
9: end function

10: function BASEREWARD( $s, t, N$ )
11:   if not HASANSWERTAG( $s$ ) then
12:     return 0
13:   else if ISCORRECT( $s, t$ ) and NODUPLICATES( $s, N$ ) then
14:     return 1
15:   else
16:     return 0.1
17:   end if
18: end function

```

4 Experimental Setup

All of the experiments were done on Amazon Web Services compute instances with NVIDIA L40S Tensor Core GPU with 48GB of GPU memory and 32GB of RAM. Our implementation used PyTorch 2.7 and was executed on Ubuntu 22.04.

4.1 Training Setup for SFT

We implemented supervised fine tuning (SFT) setup for mathematical reasoning task. As instructed, we started from Qwen 2.5 0.5B Base model Yang et al. (2024) and used train split of WarmStart dataset Asap7772 (2025) for training and test split for validation. For training strategy, we implemented next token prediction task where the model is given the prompt which is 'query' from the dataset and is trained to predict 'completion' with language model loss implemented in the Qwen model. We used AdamW Loshchilov and Hutter (2019) for the optimizer with default momentum and weight decay parameters. We trained 10 epochs with batch size of 4 with gradient accumulation of 10 steps. Validation was done every 100 iterations and we selected the model that has smallest validation loss on the validation set. We experimented with learning rate of 5×10^{-5} , 1×10^{-4} , 2×10^{-4} and 5×10^{-4} .

We used validation loss on test split of WarmStart for determining when to stop training and tested on 500 data points from Countdown dataset Pan (2025) to choose which model to submit to the leaderboard. The prompt was formatted the same way as from SFT and we utilized the scoring function from TinyZero Pan et al. (2025) with slight modification to match our output format.

4.2 Performance Improvement with RLOO

Starting from our best performing model trained with SFT, we further tune it using RLOO. For the training, we used the Countdown dataset Pan (2025) which only has the train split. Unlike the WarmStart dataset, the Countdown dataset has two labels which are number arrays and the target. For consistent setup, we constructed the same prompt as we used in SFT setting and used the same scoring function from TinyZero Pan et al. (2025) for the reward function. Due to the GPU memory limit, sample size (k) of 2 were chosen with batch size of 1. To compensate for this, we implemented gradient accumulation of 10 iterations. For additional stability, we implemented KL divergence regularization with hyperparameter β from the SFT model we started with. We experimented with learning rate and β for best performance.

As an evaluation metric, we utilized the average score of the 50 data points sampled from the Countdown dataset. We tested every 10 iteration during training.

4.3 Curriculum Learning

We extend from our RLOO implementation making changes to the original reward function as described in the Algorithm 1. We started from our best performing model from RLOO stage and further tuned with curriculum learning. For the training dataset, we also used the Countdown dataset as before but trained on the data points that was never seen before. For better stability, we used β value of 0.2 with learning rate of 4×10^{-7} , gradient accumulation of 10 steps. However, our running correctness ratio r never got past 0.4 resulting the curriculum stage of 2 and 3 to be unexplored.

To determine early stopping, we monitored average reward of the sampled responses at each iteration.

5 Results

5.1 SFT

All the experiments with different learning rates are shown in Table 1 below. Out of these experiments, we take the best performing setting with learning rate of 2×10^{-4} to test the performance on the leaderboard. The training and validation loss curve for this setting is shown below in Figure 1. Testing on the leaderboard, we achieved the score of **0.4105**.

Experiment	Learning Rate	Evaluation Score
1	5×10^{-5}	0.2610
2	1×10^{-4}	0.3120
3	2×10^{-4}	0.3240
4	5×10^{-4}	0.0870

Table 1: Results of all SFT experiments.

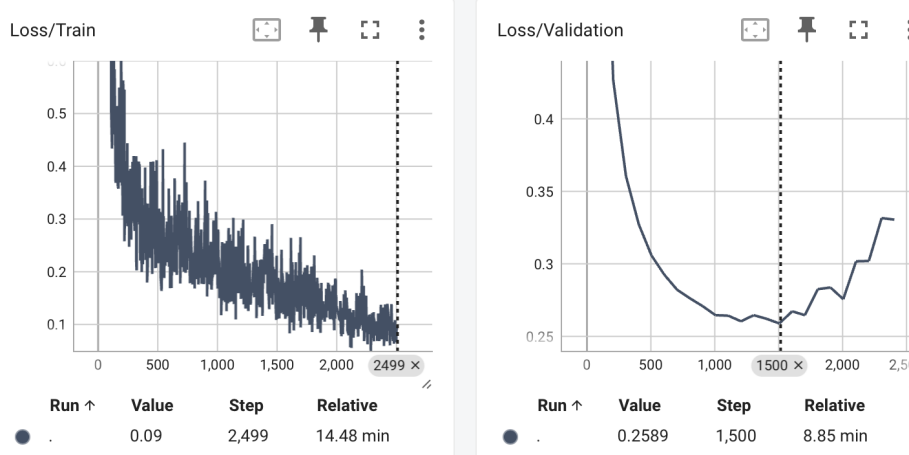


Figure 1: Training and validation loss curve of best performing setting.

5.2 RLOO

During training with RLOO, we found that stabilizing training was extremely difficult. Due to this reason, we extensively experimented various combination of learning rate and KL regularization coefficient β . In Figure 2, we show some examples of our experiments that performed reasonably well. From these settings, we selected the setting with learning rate of 8×10^{-7} and β of 0.1. After we selected the best performing model by validating on the Countdown dataset, we evaluated the performance by submitting to the leaderboard. As a result, we achieved **0.4465** which is 9% better than the SFT result on the same milestone leaderboard. We also submitted the same model to final RL leaderboard achieving **0.2620**.

5.3 Curriculum Learning

In curriculum learning experiment, we tried the same setting with RLOO experiment which is with learning rate of 8×10^{-7} and β of 0.1. However, with these hyperparameters, the average reward per iteration quickly collapsed to zero. Due to this reason, we chose 4×10^{-7} and β of 0.2 for better stability. In Figure 3 we show average reward at each iteration. Since the reward behavior is inherently noisy, we chose the iteration 140, because higher reward is achieved in the previous and future iterations with some margin around this position.

Submitting to the final RL leaderboard, we achieved **0.3079** for the performance which is 18% better than the original RLOO trained model.

We also analyzed if curriculum trained model is producing a shorter response than the RLOO trained model. To do this, we computed average length of the generated strings on the `countdown_heldout_prompts.json` and performed Wilcoxon signed rank test as shown in Table 2 below. As a result, it turns out that the average length is longer with curriculum trained model contrary to our hypothesis.

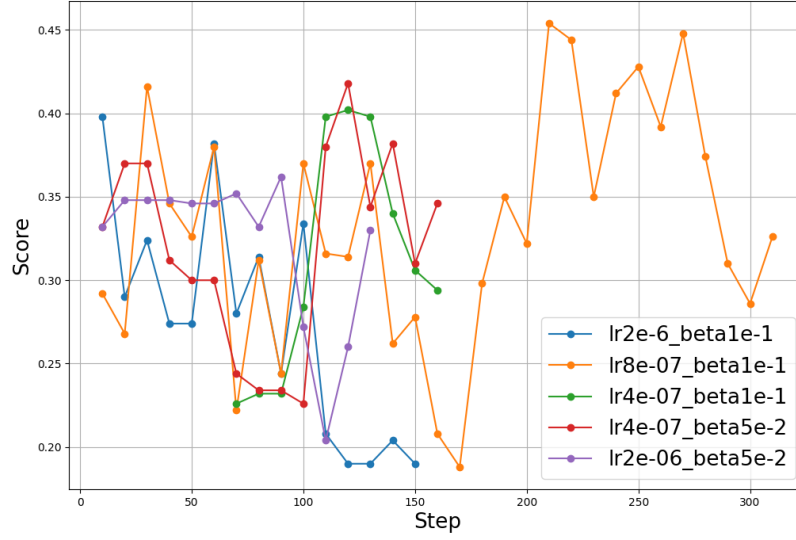


Figure 2: RLOO experiments with various hyperparameters.

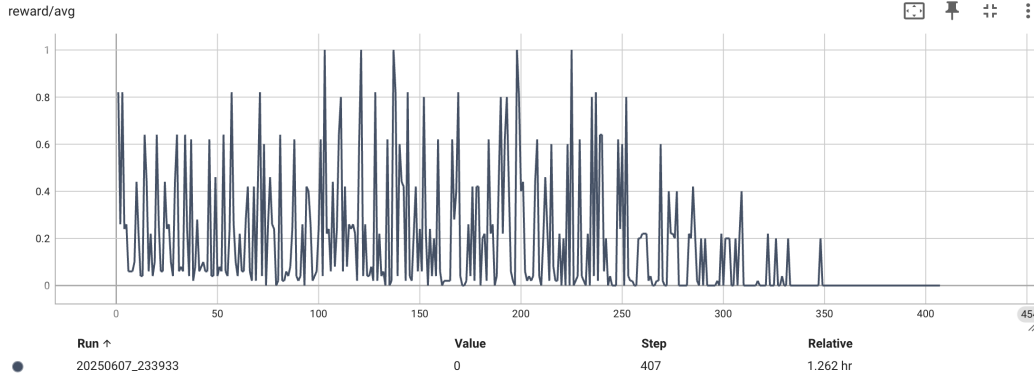


Figure 3: Average reward evaluated at each iteration during curriculum learning.

6 Discussion

6.1 Unexpected outcome

We hypothesized that rewarding brevity would shorten model outputs and yield neutral or positive effects on accuracy. Contrary to this expectation, the curriculum *increased* accuracy while *lengthening* the average chain-of-thought by roughly 150 tokens. These results show that our reward design changed behavior, but not in the direction we intended.

6.2 Why a longer CoT emerged

Two factors likely account for this divergence. First, the unit reward for a correct answer overwhelms length bonus, so gradient updates inevitably prioritize correctness at the expense of conciseness. As a result, the model *hacks* the objective by pursuing the dominant payoff and ignoring the weaker signal. Second, explicit reasoning tokens might act as scratch-pad memory for a 0.5B parameter model trying to tackle the combinatorial Countdown search. Removing that scaffold would raise

Model	Average Length	Std. Dev.	p-value
RLOO	979.0	390.1	$p \ll 0.001$
Curriculum	1130.9	409.1	

Table 2: Average string lengths for RLOO and curriculum trained models.

the risk of failure, so the optimizer settles on longer explanations that more reliably reach the target number.

6.3 Limitations

Because the running correctness ratio never exceeded 0.4, training remained in the first curriculum stage and the duplicate number penalty was never explored. In addition, RLOO sampling with $k = 2$ introduces high variance, potentially amplifying reward imbalance and making the policy especially sensitive to whichever signal is strongest.

7 Conclusion and Future Work

In this project, we proposed and evaluated Curriculum Based Reward Adjustment, a method designed to refine a fine-tuned language model for the Countdown task. The core principle was to improve task accuracy while encouraging efficiency by implementing a reward bonus for shorter, more concise solutions.

Our experiments yielded an insightful, but paradoxical outcome: while the model’s accuracy improved, its average solution length unexpectedly increased, contrary to our hypothesis. We attribute this to a form of reward hacking. The model prioritized the large, discrete reward for a correct answer over the smaller, continuous bonus for brevity. This indicates that the optimization process was dominated by the strongest signal in the reward function, leading the model to adopt a verbose but correct strategy.

This finding highlights several key directions for future work aimed at better balancing accuracy and generative efficiency. First, we plan to systematically adjust the reward hyperparameters, such as the bonus for conciseness, to find a more effective balance between the incentives for correctness and efficiency. Additionally, we will explore "harder" constraints, such as implementing a significant penalty for exceeding a length threshold, or frame the task as a multi-objective optimization problem, treating accuracy and conciseness as distinct, co-equal goals. Finally, once appropriate tuning gives us a model that reliably meets the threshold accuracy, we will retrain it with the following curriculum to further minimize its tendency to repeat numbers.

8 Team Contributions

- **Kyung Geun Kim:** Problem definition and conceptualization, Code implementation, Manuscript writing.
- **Jinho So:** Reference search, Code implementation, Manuscript writing.

Changes from Proposal

- Our initial proposal was to apply hierarchical reinforcement learning to divide the problem into subproblems with similar difficulty. However, after observing a consistent pattern of errors following RLOO stage, we pivoted our strategy. The project’s focus shifted to improving performance by directly modifying the Countdown scoring function, thereby shaping the reward model to address these specific failure modes.
- The project was completed by a team of **two members**, as one of the original members withdrew from the course.

References

- Bhavik Agarwal, Ishan Joshi, and Viktoria Rojkova. 2025. Think Inside the JSON: Reinforcement Strategy for Strict LLM Schema Adherence. arXiv:2502.14905 [cs.CL] <https://arxiv.org/abs/2502.14905>
- Asap7772. 2025. cog_behav_all_strategies. https://huggingface.co/datasets/Asap7772/cog_behav_all_strategies. Accessed: 2025-05-24.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning* (Montreal, Quebec, Canada) (ICML '09). Association for Computing Machinery, New York, NY, USA, 41–48. <https://doi.org/10.1145/1553374.1553380>
- Daniel Campos. 2021. Curriculum learning for language modeling. arXiv:2108.02170 [cs.CL] <https://arxiv.org/abs/2108.02170>
- Noelia Ferruz and Birte Höcker. 2022. Controllable protein design with language models. *Nature Machine Intelligence* 4, 6 (2022), 521–532.
- Robert M French. 1999. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences* 3, 4 (1999), 128–135.
- Hyunjae Kim, Hyeon Hwang, Jiwoo Lee, Sihyeon Park, Dain Kim, Taewhoo Lee, Chanwoong Yoon, Jiwoong Sohn, Donghee Choi, and Jaewoo Kang. 2024. Small Language Models Learn Enhanced Reasoning Skills from Medical Textbooks. arXiv:2404.00376 [cs.CL] <https://arxiv.org/abs/2404.00376>
- Jisu Kim and Juhwan Lee. 2024. Strategic Data Ordering: Enhancing Large Language Model Performance through Curriculum Learning. arXiv:2405.07490 [cs.CL] <https://arxiv.org/abs/2405.07490>
- Jean Lee, Nicholas Stevens, and Soyeon Caren Han. 2025. Large Language Models in Finance (FinLLMs). *Neural Computing and Applications* (2025), 1–15.
- Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. arXiv:1711.05101 [cs.LG] <https://arxiv.org/abs/1711.05101>
- A. Ng, Daishi Harada, and Stuart J. Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. <https://api.semanticscholar.org/CorpusID:5730166>
- Jiayi Pan. 2025. Countdown-Tasks-3to4. <https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4>. Accessed: 2025-05-24.
- Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. 2025. TinyZero. <https://github.com/Jiayi-Pan/TinyZero>. Accessed: 2025-01-24.
- Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. Progprompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 11523–11530.
- Benfeng Xu, Licheng Zhang, Zhendong Mao, Quan Wang, Hongtao Xie, and Yongdong Zhang. 2020. Curriculum Learning for Natural Language Understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 6095–6104. <https://doi.org/10.18653/v1/2020.acl-main.542>
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai

- Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).
- Xinlu Zhang, Chenxin Tian, Xianjun Yang, Lichang Chen, Zekun Li, and Linda Ruth Petzold. 2025. AlpaCare:Instruction-tuned Large Language Models for Medical Application. *arXiv:2310.14558 [cs.CL]* <https://arxiv.org/abs/2310.14558>
- Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592* (2023).